

Solution Requirements

Date	01 OCTOBER 2026
Team ID	6ab7f7bfd083b0e2ee72b13b
Project Name	LegalEase: AI-Powered Legal Document Generator
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The system should provide secure access to the LegalEase application and protect user-provided legal information and generated documents from unauthorized access.	High
2	Authorization levels	The system should provide appropriate access control so that only authorized users can access document-generation and document-management functions.	Medium

3	External interfaces	LegalEase shall integrate the Streamlit frontend, FastAPI backend, and Google Gemini AI API . The frontend sends document details such as document type, parties, terms, and dates to the FastAPI /generate endpoint, which communicates with Gemini and returns the generated legal content.	High
4	Transactions processing	The system shall accept the document type, parties involved, terms and conditions, and effective date, validate the input, send it to the Gemini AI model through FastAPI, generate the legal document, and return the result for preview and editing.	High

5	Reporting	The system shall provide a formatted preview of the generated legal document and allow users to download the final document in TXT, DOCX, and PDF formats. DOCX/PDF outputs should maintain appropriate formatting, branding, headings, tables, logos, and footers.	High
6	Business rules, etc...	The system shall generate documents according to the selected document type and user-provided parties, terms, and effective date. Semicolon-separated terms shall be converted into structured clauses or bullet points. Users shall also be able to edit the generated content before downloading it.	High
7	Compliance to laws or regulations	The system should clearly indicate that AI-generated legal content should be reviewed before official use. User-provided legal information should be handled securely, and the system should avoid presenting AI-generated content as a substitute for professional legal advice.	High

8	Other	The system shall provide a simple, responsive and user-friendly interface that enables users, including those without a legal background, to enter legal details, generate documents, preview them, edit them, and download the final document in the required format.	Medium
----------	-------	--	--------

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The system should generate and display AI-powered legal documents efficiently after the user submits the document type, parties, terms, and effective date.	Document generation and preview should normally be completed within 10 seconds , excluding delays from the external Gemini API. The interface should respond without unnecessary delays during input and editing.
2	Scalability	The system should support an increasing number of users and document-generation requests without requiring major changes to the Streamlit frontend, FastAPI backend, or AI integration architecture.	The architecture should support multiple concurrent document-generation requests and allow additional backend resources to be added when demand increases.
3	Security & Data Privacy	User-provided legal information, generated documents, and API credentials should be protected. Sensitive credentials such as the Gemini API key should not be exposed in the source code and should be stored using environment variables.	API keys are stored in .env/environment variables; no credentials are hard-coded; communication with backend/API services is protected; unauthorized users cannot access protected data. Your project documentation explicitly specifies storing the Gemini API key securely in a .env file.

4	Reliability & Availability	The system should reliably process user requests and handle failures in the FastAPI backend, Gemini API integration, or document-generation modules without crashing the entire application.	Successful processing for valid inputs; invalid or failed requests should produce a meaningful error message. The FastAPI backend should remain available during normal operation and all major generation/export functions should work correctly.
5	Usability & Accessibility	The application should provide a simple and user-friendly interface that allows users, including users without a legal background, to enter details, generate documents, preview them, edit them, and download them.	A user should be able to complete the Input → Generate → Preview → Edit → Download workflow without technical knowledge. The interface should clearly identify document type, parties, terms, effective date, and download options..
6	Maintainability & Portability	The system should use a modular architecture separating the Streamlit frontend, FastAPI backend, Gemini AI integration, and document-formatting modules. It should be easy to install and run in another supported environment	Application components remain independently maintainable; dependencies are defined in requirements.txt; environment-specific configuration is externalized; the application can be set up using the documented Python environment and dependencies.